

Experiment No.11

Aim: Building a text-based DS Application on Sentiment Analysis

- 1) Understand the different data science phases used in Sentiment Analysis
- 2) Implementation of Machine Learning algorithm for Sentiment Analysis

Theory:

Natural language processing (NLP):

Natural language processing (NLP) is a subfield of Artificial Intelligence (AI). This is a widely used technology for personal assistants that are used in various business fields/areas. This technology works on the speech provided by the user breaks it down for proper understanding and processes it accordingly. This is a very recent and effective approach due to which it has a really high demand in today's market. Natural Language Processing is an upcoming field where already many transitions such as compatibility with smart devices, and interactive talks with a human have been made possible. Knowledge representation, logical reasoning, and constraint satisfaction were the emphasis of AI applications in NLP. Here first it was applied to semantics and later to grammar. In the last decade, a significant change in NLP research has resulted in the widespread use of statistical approaches such as machine learning and data mining on a massive scale. The need for automation is never-ending courtesy of the amount of work required to be done these days. NLP is a very favourable, but aspect when it comes to automated applications. The applications of NLP have led it to be one of the most sought-after methods of implementing machine learning. Natural Language Processing (NLP) is a field that combines computer science, linguistics, and machine learning to study how computers and humans communicate in natural language. The goal of NLP is for computers to be able to interpret and generate human language. This not only improves the efficiency of work done by humans but also helps in interacting with the machine. NLP bridges the gap of interaction between humans and electronic devices.

Natural Language Processing (NLP) is a subfield of artificial intelligence that deals with the interaction between computers and humans in natural language. It involves the use of computational techniques to process and analyse natural language data, such as text and speech, with the goal of understanding the meaning behind the language.

NLP is used in a wide range of applications, including machine translation, sentiment analysis, speech recognition, chatbots, and text classification. Some common techniques used in NLP include:

1. **Tokenization:** the process of breaking text into individual words or phrases.
2. **Part-of-speech tagging:** the process of labelling each word in a sentence with its grammatical part of speech.
3. **Named entity recognition:** the process of identifying and categorizing named entities, such as people, places, and organizations, in text.
4. **Sentiment analysis:** the process of determining the sentiment of a piece of text, such as whether it is positive, negative, or neutral.
5. **Machine translation:** the process of automatically translating text from one language to another.
6. **Text classification:** the process of categorizing text into predefined categories or topics.

NLP Tasks:

Several NLP tasks break down human text and voice data in ways that help the computer make sense of what it's ingesting. Some of these tasks include the following:

- **Speech recognition**, also known as speech-to-text, is a challenging task that involves converting voice data into text data. This technology is essential for any application that requires voice commands or spoken responses. However, people's speaking habits, such as speaking quickly, slurring words, using different accents, and incorrect grammar, make speech recognition even more challenging.
- **Part of speech tagging**, also known as grammatical tagging, is a crucial process that determines the part of speech of a specific word or piece of text based on its context and usage. For example, it can identify 'make' as a verb in 'I can make a paper plane' and as a noun in 'What make of car do you own?'
- **Word sense disambiguation** is a semantic analysis process that selects the most appropriate meaning of a word with multiple meanings based on the given context. This process is helpful in distinguishing the meaning of the verb 'make' in 'make the grade' (achieve) versus 'make a bet' (place).
- **Named entity recognition (NEM)** identifies useful entities or phrases, such as 'Kentucky' as a location or 'Fred' as a person's name. Co-reference resolution is the task of identifying when two words refer to the same entity, such as determining that 'she' refers to 'Mary.' Sentiment analysis is a process that attempts to extract subjective qualities, including attitudes, emotions, sarcasm, confusion, and suspicion, from text.

- **Natural language generation** is the opposite of speech recognition, as it involves putting structured information into human language. Overall, understanding these processes is essential in building effective natural language processing systems.

Applications of Natural Language Processing (NLP):

- **Spam Filters:** One of the most irritating things about email is spam. Gmail uses natural language processing (NLP) to discern which emails are legitimate and which are spam. These spam filters look at the text in all the emails you receive and try to figure out what it means to see if it's spam or not.
- **Algorithmic Trading:** Algorithmic trading is used for predicting stock market conditions. Using NLP, this technology examines news headlines about companies and stocks and attempts to comprehend their meaning in order to determine if you should buy, sell, or hold certain stocks.
- **Questions Answering:** NLP can be seen in action by using Google Search or Siri Services. A major use of NLP is to make search engines understand the meaning of what we are asking and generate natural language in return to give us the answers.
- **Summarizing Information:** On the internet, there is a lot of information, and a lot of it comes in the form of long documents or articles. NLP is used to decipher the meaning of the data and then provides shorter summaries of the data so that humans can comprehend it more quickly.

Methodology:

Roberta Pretrained Model:

RoBERTa, being a highly capable transformer-based language model, is also extensively used in sentiment analysis tasks due to its ability to understand contextual information and capture nuanced linguistic features. Here's how RoBERTa pretrained models are typically utilized in sentiment analysis:

Fine-tuning:

-  Pretrained RoBERTa models are fine-tuned on sentiment analysis datasets to adapt them to the specific task of sentiment classification.
-  During fine-tuning, the RoBERTa model is initialized with pretrained weights and then trained on a labelled sentiment analysis dataset.
-  The model learns to associate contextual information in the text with sentiment labels (positive, negative, neutral) through backpropagation and gradient descent optimization.

Tokenization:

- Text data is tokenized into subword tokens using the same tokenization scheme used during RoBERTa's pretraining.
- RoBERTa uses byte pair encoding (BPE) tokenization, which splits words into subword units based on their frequency in the training corpus.
- Tokenized text is then converted into numerical representations that can be fed into the RoBERTa model.

Input Encoding:

- The tokenized text is encoded into input embeddings using the RoBERTa model's embedding layer.
- These embeddings capture the semantic meaning and contextual information of the input text, which is crucial for sentiment analysis.

Model Training:

- The encoded input embeddings are passed through multiple layers of RoBERTa's transformer architecture.
- The model learns to attend to relevant parts of the input text and generate representations that are useful for predicting sentiment labels.
- During training, the model's weights are updated to minimize a loss function, such as cross-entropy loss, which measures the difference between predicted and true sentiment labels.

Evaluation:

- The fine-tuned RoBERTa model is evaluated on a separate validation or test dataset to assess its performance on sentiment analysis.
- Evaluation metrics such as accuracy, precision, recall, F1-score, or area under the ROC curve (AUC-ROC) are used to measure the model's effectiveness in predicting sentiment labels.

Inference:

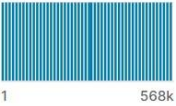
- Once trained and evaluated, the fine-tuned RoBERTa model can be used to predict sentiment labels for new, unseen text data.
- Text data is tokenized, encoded, and passed through the model, which generates predictions for the sentiment of each input text.

Overall, RoBERTa pretrained models offer a powerful and effective solution for sentiment analysis tasks, providing accurate predictions and capturing subtle nuances in text sentiment. By fine-tuning RoBERTa on sentiment analysis datasets, researchers and practitioners can leverage its advanced capabilities to extract valuable insights from text data.

Implementation:

1. Data Collection:

- Identify relevant sources: Determine where the opinions you're interested in are being expressed. This could be social media platforms like Twitter, review websites like Yelp or Amazon, forums, or news articles.
- Collect diverse data: Ensure that your dataset covers a wide range of opinions and sentiments to train a robust sentiment analysis model. Include data from different demographics, geographic locations, and time periods if possible.
- Consider data quality: Ensure the data you collect is of high quality, free from noise, and relevant to your analysis task. Low-quality data can lead to biased or inaccurate sentiment analysis results.

Reviews.csv (300.9 MB)						↓ >
Detail Compact Column						10 of 10 columns ▾
About this file						
All data in one sqlite database. 568,454 food reviews Amazon users left up to October 2012						
Id Row Id	ProductId Unique identifier for the product	UserId Unique identifier for the user	ProfileName Profile name of the user	# HelpfulnessNum... Number of users who found the review helpful	# H Num indic foun or nc	
	74258 unique values	256059 unique values	218418 unique values			

Input (673.7 MB)

- Data Sources
 - Amazon Fine Food Review
 - Reviews.csv
 - database.sqlite
 - hashes.txt

2. Data Preprocessing:

- Text cleaning: Remove irrelevant information such as HTML tags, special characters, URLs, and punctuation marks from the text data.
- Tokenization: Split the text into individual words, phrases, or tokens. This step makes it easier to analyse and process the text data.
- Normalization: Convert all text to a consistent format, such as lowercase, to ensure uniformity in the dataset. Additionally, perform tasks like stemming or lemmatization to reduce words to their base forms.
- Stop word removal: Eliminate common words like "and", "the", "is" that do not carry significant meaning for sentiment analysis and could introduce noise into the analysis.

[5]:		ID	ProductID	UserID	ProfileName	HelpfulnessNumerator	HelpfulnessDenominator	Score	Time	Summary	Text
	0	1	B001E4KFG0	A3SGXH7AUHU8GW	delmartian	1	1	5	1303862400	Good Quality Dog Food	I have bought several of the Vitality canned d...
	1	2	B00813GRG4	A1D87F6ZCVE5NK	dll pa	0	0	1	1346976000	Not as Advertised	Product arrived labeled as Jumbo Salted Peanut...

- **Bag-of-Words (BoW):** Represent text data as a numerical vector, where each dimension corresponds to a unique word in the vocabulary, and the value represents the frequency of that word in the document.
- **TF-IDF (Term Frequency-Inverse Document Frequency):** Similar to BoW, but it also considers the importance of words in the context of the entire dataset. Words that are frequent in a document but rare in the overall corpus are weighted more heavily.
- **Word Embeddings:** Represent words as dense, low-dimensional vectors that capture semantic meaning. Pre-trained word embeddings like Word2Vec, Glove, or BERT embeddings can be used to convert text data into numerical representations.
- **n-grams:** Capture the co-occurrence of adjacent words or characters in the text, providing additional context for sentiment analysis.

```
[15]: import nltk
      nltk.download('vader_lexicon')

[nltk_data] Downloading package vader_lexicon to
[nltk_data] C:\Users\meets\AppData\Roaming\nltk_data...
[nltk_data] Package vader_lexicon is already up-to-date!

[15]: True

[16]: from nltk.sentiment import SentimentIntensityAnalyzer
      from tqdm.notebook import tqdm

      sia = SentimentIntensityAnalyzer()

[17]: sia.polarity_scores('I am so happy!')

[17]: {'neg': 0.0, 'neu': 0.318, 'pos': 0.682, 'compound': 0.6468}

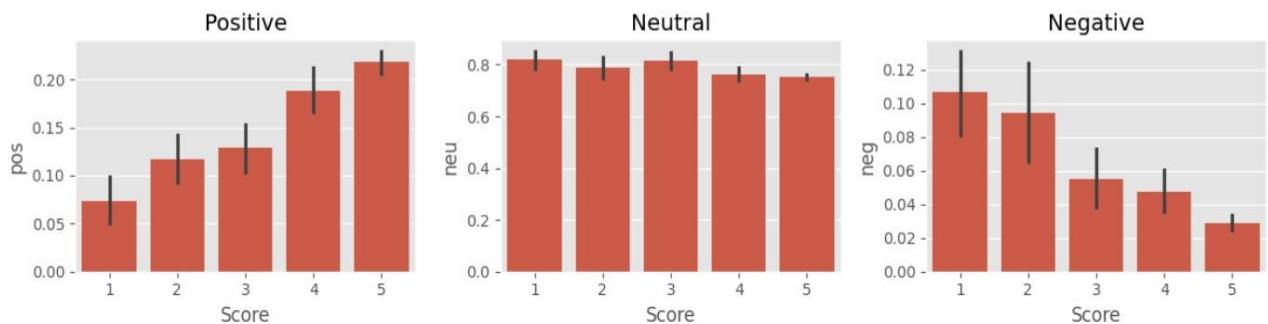
[18]: sia.polarity_scores('This is the worst thing ever.')

[18]: {'neg': 0.451, 'neu': 0.549, 'pos': 0.0, 'compound': -0.6249}

[19]: sia.polarity_scores(example)

[19]: {'neg': 0.22, 'neu': 0.78, 'pos': 0.0, 'compound': -0.5448}
```

```
[24]: fig, axs = plt.subplots(1, 3, figsize=(12, 3))
sns.barplot(data=vaders, x='Score', y='pos', ax=axs[0])
sns.barplot(data=vaders, x='Score', y='neu', ax=axs[1])
sns.barplot(data=vaders, x='Score', y='neg', ax=axs[2])
axs[0].set_title('Positive')
axs[1].set_title('Neutral')
axs[2].set_title('Negative')
plt.tight_layout()
plt.show()
```



4. Model Training:

- **Select a suitable model:** Choose a machine learning or deep learning algorithm for sentiment analysis based on factors such as dataset size, complexity, and computational resources available.
- **Data splitting:** Divide the dataset into training, validation, and test sets to train the model, tune hyperparameters, and evaluate performance, respectively.
- **Train the model:** Feed the pre-processed text data into the chosen model and adjust model parameters to minimize the loss function, optimizing the model for sentiment analysis.
- **Cross-validation:** Perform k-fold cross-validation to assess the generalization performance of the model and ensure that it's not overfitting to the training data.

```
[33]: from transformers import AutoTokenizer, AutoModelForSequenceClassification

MODEL = "cardiffnlp/twitter-roberta-base-sentiment"
tokenizer = AutoTokenizer.from_pretrained(MODEL)
model = AutoModelForSequenceClassification.from_pretrained(MODEL)
```

```
from nltk.sentiment import SentimentIntensityAnalyzer
```

Define an example text

```
example = "This oatmeal is not good. Its mushy, soft, I don't like it. Quaker Oats is the way to go."
```

Create a SentimentIntensityAnalyzer object

```
sia = SentimentIntensityAnalyzer()
```

Get the polarity scores for the example text

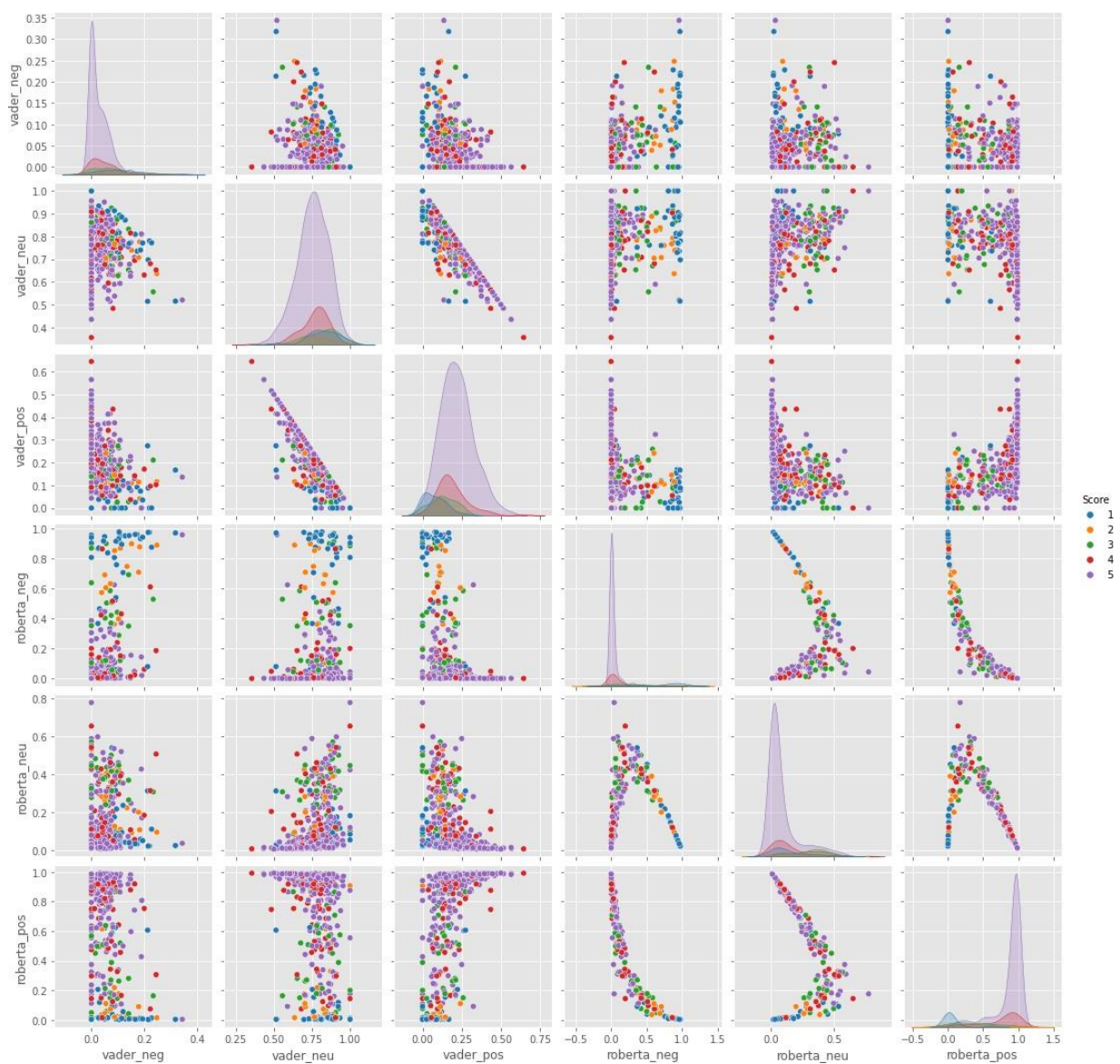
```
polarity_scores = sia.polarity_scores(example)
```

Print the example text and its polarity scores

```
print("Example Text:", example) print("Polarity Scores:", polarity_scores)
```


5. Model Evaluation:

- Evaluate metrics: Measure the performance of the trained model using evaluation metrics such as accuracy, precision, recall, F1-score, or area under the ROC curve (AUC-ROC).
- Confusion matrix: Analyze the distribution of true positive, true negative, false positive, and false negative predictions to gain insights into the model's strengths and weaknesses.
- Error analysis: Examine misclassified instances to identify patterns or recurring issues that may indicate areas for improvement in the model or data preprocessing pipeline.



6. Model Deployment:

- If the model meets the desired performance criteria and is deemed suitable for production use, deploy it into a production environment where it can be used to analyze real-time text data.
- Implement APIs or integrations: Develop interfaces or integrations that allow the sentiment analysis model to be seamlessly integrated with existing systems or applications.

In [32]:

```
from transformers import pipeline  
  
sent_pipeline = pipeline("sentiment-analysis")
```

No model was supplied, defaulted to distilbert-base-uncased-finetuned-sst-2-english (<https://huggingface.co/distilbert-base-uncased-finetuned-sst-2-english>)

Downloading: 100%  629/629 [00:00<00:00, 20.9kB/s]

Downloading: 100%  255M/255M [00:12<00:00, 23.0MB/s]

Downloading: 100%  48.0/48.0 [00:00<00:00, 1.42kB/s]

Downloading: 100%  226k/226k [00:00<00:00, 897kB/s]

7. Monitoring and Maintenance:

- Monitor model performance: Continuously monitor the performance of the deployed model in production to ensure that it maintains its accuracy and effectiveness over time.
- Update model as needed: Retrain the model periodically with new data to adapt to changes in data distribution, user behaviour, or language trends.

Result:

```
[51]: sent_pipeline('I absolutely loved my experience at the restaurant. The ambiance was delightful, the service was impeccable, and the food was simply divin
```

```
[51]: [{'label': 'POSITIVE', 'score': 0.9998865127563477}]
```

```
[52]: sent_pipeline('My experience at the hotel was terrible. The room was dirty, the staff was rude, and the overall atmosphere was unpleasant. I would not re
```

```
[52]: [{'label': 'NEGATIVE', 'score': 0.9994731545448303}]
```

```
[ ]:
```

Compare Scores between models

```
[40]: results_df.columns

[40]: Index(['Id', 'vader_neg', 'vader_neu', 'vader_pos', 'vader_compound',
        'roberta_neg', 'roberta_neu', 'roberta_pos', 'ProductId', 'UserId',
        'ProfileName', 'HelpfulnessNumerator', 'HelpfulnessDenominator',
        'Score', 'Time', 'Summary', 'Text'],
        dtype='object')

[41]: results_df.query('Score == 1') \
      .sort_values('roberta_pos', ascending=False)['Text'].values[0]

[41]: 'Product arrived labeled as Jumbo Salted Peanuts...the peanuts were actually small sized unsalted. Not sure if this was an error or if the vendor intended to represent the product as "Jumbo".'
```

```
[42]: results_df.query('Score == 1') \
      .sort_values('vader_pos', ascending=False)['Text'].values[0]

[42]: 'Quality of product is excellent, cost is excellent and delivery is superb. Very satisfied.'
```

```
[43]: # nevative sentiment 5-Star view

[44]: results_df.query('Score == 5') \
      .sort_values('roberta_neg', ascending=False)['Text'].values[0]

[44]: 'I have bought several of the Vitality canned dog food products and have found them all to be of good quality. The product looks more like a stew than a processed meat and it smells better. My Labrador is finicky and she appreciates this product better than most.'
```

Conclusion:

In conclusion, sentiment analysis serves as a crucial tool in understanding and extracting insights from vast amounts of text data across diverse domains. By leveraging advanced natural language processing techniques and machine learning algorithms, sentiment analysis enables businesses, researchers, and decision-makers

- ✚ Gain valuable insights into customer opinions, preferences, and satisfaction levels.
- ✚ Monitor brand reputation and sentiment trends in real-time across various platforms and channels.
- ✚ Identify emerging issues, sentiment shifts, or potential crises early on, allowing for proactive intervention.
- ✚ Personalize marketing campaigns, product recommendations, and customer interactions based on individual sentiment profiles.
- ✚ Enhance product development processes by analysing user feedback and sentiment to prioritize features and improvements.
- ✚ Inform strategic decision-making and resource allocation based on sentiment analysis-driven insights.

Overall, sentiment analysis empowers organizations to harness the power of language data to drive informed decisions, foster customer loyalty, and stay ahead in today's dynamic and competitive landscape. As technologies like RoBERTa continue to advance, sentiment analysis will play an increasingly integral role in shaping business strategies, optimizing user experiences, and unlocking new opportunities for innovation and growth.